Figure 14.10 BFS in action. In this example, we show a single-source, all-vertices run: computing the distance between a source and all vertices in the graph. The algorithm explicitly maintains a queue with the vertices in the "frontier", which will be explored next. The vertices are naturally ordered by their distance from the source, so a regular queue can be used, though conceptually the algorithm behaves as if it was using a priority queue.

So, besides the initialization performed in lines #2-#7, the core of the BFS algorithm is the loop at line #8: a vertex v is dequeued and visited; after (optionally) checking if we reached the search's goal (if so, we can already return v), we start exploring v's neighborhood (aka its adjacency list, all its outgoing edges): if we find a vertex u that we hadn't discovered yet, then we know that its distance from source will be the distance to get to v plus 1, therefore we can set the distance for u and add u to the tail of the queue.

How can we be sure that u is going to be visited in the right order, i.e. after all vertices closer to the source than u, and before the ones further away than u?

This can be proved by induction on the distance of vertices. The base of the induction is the initial case: we add the source vertex with a distance `0`; this vertex is the only one at distance `0`, and so any other vertex added later can't be closer (or as close as) than the source.

For the inductive step, we assume that our hypothesis is true for all vertices at distance `d-1`, and we want to prove it for